



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE CODE: CSA1407

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 10%

QUESTIONS:

Total Marks: 50

Annexure A

SCENARIO BASED TASK

Code of Conduct:

I Juhi Surin (192525148) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Juhi Surin



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



Annexure A

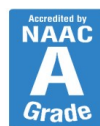
SCENARIO-BASED TASK

QUESTIONS:

- 1.Scenario: Parsing Table Conflict in LL(1) While designing an LL(1) parser, a student constructs a parsing table and finds multiple entries in a single cell for a non-terminal and input symbol combination. This leads to confusion during parsing. Analyze how the syntax analyzer uses FIRST and FOLLOW sets in this process, identify the reason for the conflict, and explain how it affects deterministic parsing. Conclude by suggesting what decision should be taken to modify the grammar so that it becomes suitable for LL(1) parsing.
- 2.Scenario: Incorrect Parse Tree Generation A syntax analyzer generates a parse tree for the expression $id + id - id$, but the evaluation gives incorrect results due to improper grouping of operators. Analyze how the syntax analyzer constructs the parse tree, identify the issue related to associativity, and explain how grammar rules influence the structure of the tree. Finally, discuss what decision should be taken to enforce correct associativity and ensure accurate parsing.
- 3.Scenario: Invalid Expression Handling A syntax analyzer in a compiler receives the token sequence corresponding to the expression $a + * b$ from the lexical analyzer. Although all tokens are valid individually, the parser fails to generate a parse tree and reports an error. In this situation, analyze how the syntax analyzer identifies the error based on grammar rules, determine the key issue in the token arrangement, and explain how parsing techniques such as predictive or shift-reduce parsing handle such cases. Further, discuss what decision should be taken to improve error detection and recovery while maintaining clarity in parsing.
- 4.Scenario: Ambiguity in Conditional Statements A syntax analyzer is designed using the grammar $S \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S \mid a$. While constructing the parsing table, conflicts arise, and the parser cannot decide which production to apply for certain inputs. In this scenario, explain how the syntax analyzer encounters ambiguity, identify the key issue known as the dangling else problem, and discuss how it impacts parsing. Suggest an appropriate modification or parsing strategy to resolve the ambiguity and ensure correct association of else clauses.
- 5.Scenario: Operator Precedence Conflict A parser is designed for arithmetic expressions using the grammar $E \rightarrow E + E \mid E * E \mid id$. While parsing the input $id + id * id$, the syntax analyzer produces an incorrect parse tree due to ambiguity in operator precedence. Examine how the syntax analyzer processes this input, identify the issue caused by the grammar, and explain how precedence and associativity rules should be applied. Based on this, suggest what changes must be made to the grammar or parsing technique to ensure correct evaluation.



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



RUBRICS FOR CALCULATION:

Scenario based assessment

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand



1. Scenario: Parsing Table Conflict in LL(1)

Understanding the Scenario

An LL(1) parser relies on a predictive parsing table indexed by (non-terminal, input symbol) pairs, and each cell must hold exactly one production for the parser to make a deterministic choice while scanning the input from left to right with one symbol of lookahead. When the student finds two productions occupying the same cell, the table itself has become ambiguous with respect to that grammar, and the parser has no rule for choosing between the alternatives at that point in the derivation.

Role of FIRST and FOLLOW Sets

The table is built by computing, for every production $A \rightarrow \alpha$, the set $\text{FIRST}(\alpha)$; the production is entered in row A and column t for every terminal t in $\text{FIRST}(\alpha)$. If α can derive ϵ , the production is additionally entered in row A for every terminal in $\text{FOLLOW}(A)$, since the parser must know when it is safe to skip A and move on. A collision in a single cell therefore means that two productions of the same non-terminal share a terminal in their FIRST sets, or that an ϵ -derivable alternative's FOLLOW set overlaps with another alternative's FIRST set.

Key Issue Behind the Conflict

Two grammar constructions commonly cause this overlap:

- Left recursion, where a non-terminal such as $A \rightarrow A\alpha \mid \beta$ keeps re-deriving itself, so the parser cannot decide in one lookahead symbol whether to expand or terminate the recursion.
- Common prefixes between alternatives (the left-factoring problem), where two productions of the same non-terminal begin with the same terminal or non-terminal, so FIRST sets intersect and the parser cannot tell which alternative to expand from the first symbol alone.

Effect on Deterministic Parsing

LL(1) parsing depends on determinism: exactly one action per cell. A cell with two entries forces the parser to guess, which either causes a runtime failure, an incorrect derivation, or requires backtracking — all of which defeat the purpose of a single-lookahead top-down parser and make the grammar unfit for standard LL(1) tools.

Decision

The grammar should be transformed before it is handed to the parser generator: eliminate left recursion by converting $A \rightarrow A\alpha \mid \beta$ into $A \rightarrow \beta A'$, $A' \rightarrow \alpha A' \mid \epsilon$, and apply left factoring so that productions sharing a prefix are rewritten as $A \rightarrow \delta A'$, $A' \rightarrow \alpha_1' \mid \alpha_2'$. After both transformations, FIRST sets of the alternatives of each non-terminal become disjoint (and FOLLOW sets are checked for ϵ -derivable cases), which yields a conflict-free LL(1) parsing table and restores deterministic, single-lookahead parsing.

2. Scenario: Incorrect Parse Tree Generation

Understanding the Scenario

The parse tree for $\text{id} + \text{id} - \text{id}$ is built by repeatedly applying grammar productions to the input tokens until every leaf is a terminal. The shape of that tree — specifically, which operator's subtree encloses the other — determines the order of evaluation, so an incorrectly shaped tree changes the computed result even though every individual token was recognized correctly.

Key Issue: Associativity

The evaluator got the wrong answer because the grammar used to build the tree did not fix an associativity direction for $+$ and $-$. Without that constraint, the same token stream can be grouped as $\text{id} + (\text{id} - \text{id})$ or as $(\text{id} + \text{id}) - \text{id}$, and only the second grouping matches the mathematically correct left-to-right evaluation of same-precedence



operators. The parser essentially had two legal ways to nest the operators, and it picked the one that groups the trailing operand with the wrong operator.

How Grammar Rules Shape the Tree

A production such as $E \rightarrow E + E \mid E - E \mid id$ is ambiguous for this exact reason: it does not say which E should be built first, so a parser (or a human tracing the derivation) can justify more than one tree for the same string. The structure of the tree is entirely a product of how the grammar is written, not of the input alone — so fixing the tree means fixing the grammar.

Decision

Left associativity should be encoded directly into the grammar by making the non-terminal recurse on its left side only:

- $E \rightarrow E + T \mid E - T \mid T$
- $T \rightarrow id$

With this rewrite, the derivation for $id + id - id$ can only nest as $((id + id) - id)$, because the recursive E is always the left operand. This removes the structural ambiguity, forces left-to-right grouping, and guarantees that the evaluator produces the arithmetically correct result every time.

3. Scenario: Invalid Expression Handling

Understanding the Scenario

The lexical analyzer has already done its job correctly: a , $+$, $*$, and b are each valid tokens on their own. The failure happens one stage later, in the syntax analyzer, which checks whether the sequence of tokens as a whole matches any legal derivation of the grammar — lexical validity of individual tokens does not guarantee syntactic validity of their arrangement.

Key Issue in the Token Arrangement

In a typical expression grammar, every binary operator must be followed by an operand (an identifier, constant, or parenthesized sub-expression), never by another operator. In $a + * b$, the $+$ is immediately followed by $*$ instead of an operand, so there is no production the parser can apply to continue the derivation at that point. The token stream violates the grammar's structural rules even though each token is individually legitimate.

How Parsing Techniques Detect the Error

In predictive (LL) parsing, the parser consults the parsing table for the current non-terminal and lookahead symbol; when it reaches the point expecting an operand and sees $*$ instead, no table entry exists for that combination, and the parser reports a syntax error at that exact position.

In shift-reduce (LR) parsing, the parser maintains a stack and looks for a valid shift or reduce action for the current state and input symbol; here, no shift action accepts $*$ after a $+$ has just been shifted and no reduction applies, so the parser signals an error because the action table has no defined move.

Decision

Rather than aborting immediately, the parser should be equipped with error recovery so that analysis of the remaining program can continue and multiple errors can be reported in a single pass. Suitable techniques include:

- Panic-mode recovery — discard input tokens until a synchronizing token (such as $;$ or the FOLLOW set of the current non-terminal) is found, then resume parsing.



- Phrase-level recovery — locally replace, insert, or delete a token (e.g., delete the stray '*') to allow parsing to proceed, based on common error patterns.
- FOLLOW-set based synchronization — use the FOLLOW set of the non-terminal being parsed to decide which tokens are safe re-entry points after an error.

Adopting one of these strategies keeps the compiler's error messages meaningful and precise while allowing it to keep scanning the rest of the source instead of stopping at the very first fault.

4. Scenario: Ambiguity in Conditional Statements

Understanding the Scenario

The grammar $S \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S \mid a$ allows an if-then-else construct to be nested inside a plain if-then construct. When the input contains two if's followed by a single else, the grammar does not specify which if the else is meant to close, so more than one parse tree — and therefore more than one program meaning — is possible for the same source text.

Key Issue: The Dangling Else Problem

Both productions share the common prefix 'if E then S', so after the parser has read that prefix and then encounters 'else', it cannot decide whether the else belongs to the innermost pending if or an outer one. This is the classic dangling-else ambiguity: the grammar is structurally ambiguous because it permits two distinct derivation trees for a single valid input string.

Impact on Parsing

When the parsing table is built, the cell for (S, else) receives two candidate productions — one that reduces the inner if-then as complete and one that continues to absorb the else into it — creating a shift-reduce style conflict. Because the parser cannot make a deterministic choice, either the parser generator must impose a default rule or the grammar must be rewritten, or the resulting program's control flow becomes unpredictable and compiler-dependent.

Decision

The ambiguity can be removed structurally by splitting statements into 'matched' (fully closed if-else pairs) and 'unmatched' (an if with no corresponding else yet) categories:

- $S \rightarrow M \mid U$
- $M \rightarrow \text{if } E \text{ then } M \text{ else } M \mid a$
- $U \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } M \text{ else } U$

This rewrite forces every else to attach only to a matched, fully resolved if, eliminating the double derivation. Alternatively, most real parser generators sidestep the grammar rewrite by hard-coding the convention that else binds to the nearest unmatched if, which is the de facto rule adopted by languages such as C, Java, and Python-style conditionals, and it produces the same unambiguous association without expanding the grammar.

5. Scenario: Operator Precedence Conflict

Understanding the Scenario

The grammar $E \rightarrow E + E \mid E * E \mid \text{id}$ treats addition and multiplication as structurally identical alternatives of the same non-terminal, with no rule distinguishing which operator should bind more tightly. For the input $\text{id} + \text{id} * \text{id}$, this symmetry allows the parser to justify grouping either the '+' or the '*' first, producing two different parse trees from one grammar.



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



Key Issue Caused by the Grammar

Because both operators are generated at the same level of the grammar, the derivation can proceed as $(id + id) * id$ or as $id + (id * id)$, and the grammar offers no mechanism — no precedence, no distinct non-terminal levels — to prefer one over the other. Mathematically, multiplication must bind tighter than addition, but the grammar as given encodes no such preference, so the parser's choice is arbitrary rather than rule-based.

Applying Precedence and Associativity

Correct evaluation requires two properties to be built into the grammar: a precedence order (multiplication above addition) and an associativity direction (left-to-right for both, since repeated $+$ or repeated $*$ should group from the left). Precedence is enforced by giving each operator its own non-terminal layer, with tighter-binding operators placed closer to the leaves of the grammar; associativity is enforced by left-recursive productions at each layer, exactly as in question 2.

Decision

The grammar should be layered by precedence as follows:

- $E \rightarrow E + T \mid T$
- $T \rightarrow T * F \mid F$
- $F \rightarrow id$

Here, T (term) sits below E (expression), so multiplication is always reduced before addition is applied around it, and the left recursion at every level keeps same-precedence operators left-associative. For $id + id * id$, this grammar can only derive $id + (id * id)$, which matches standard arithmetic rules. Where rewriting the grammar into layers is inconvenient, parser generators such as Yacc/Bison offer precedence and associativity declarations (e.g., `%left`, `%right`) that let the tool resolve the same conflicts automatically without changing the grammar's shape.